

Cloud Specialist
Unità Formativa (UF): Containers Microservizi
Serverless
Docente: Denis Maggiorotto
Titolo argomento: FS Layering e Volumi

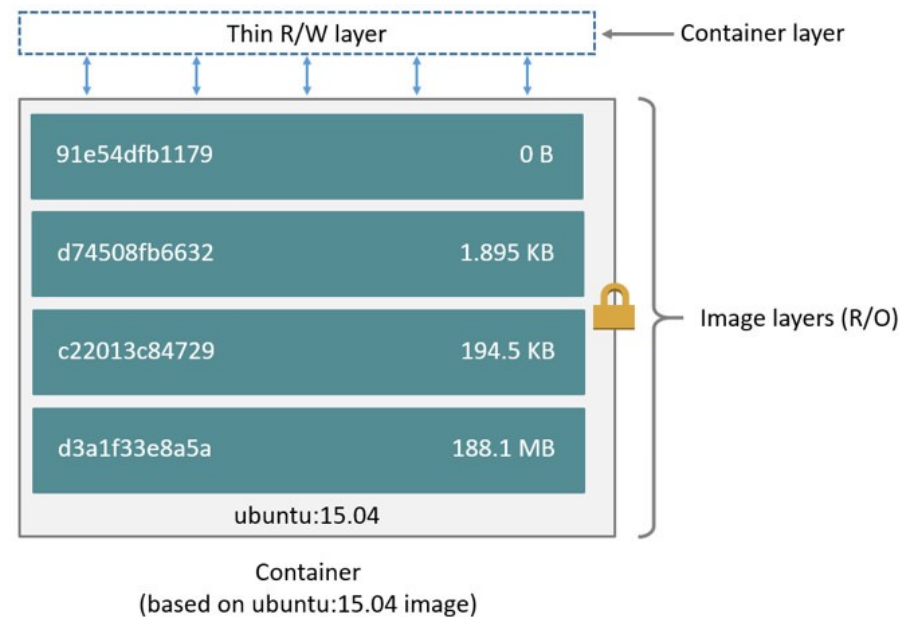
File System layering



Containers layers

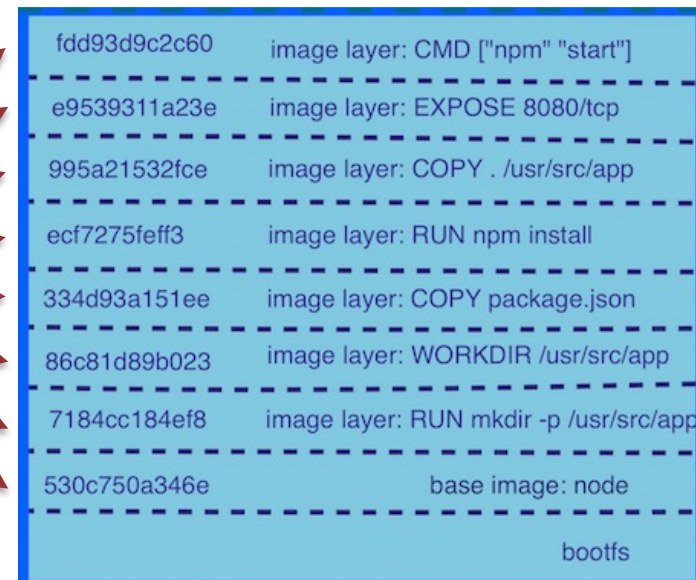
I containers (e le immagini) sono composti da layers (livelli).

I livelli delle immagini sono Read Only (RO), il livello dei container è l'unico Read/Write (RW)



Containers layers

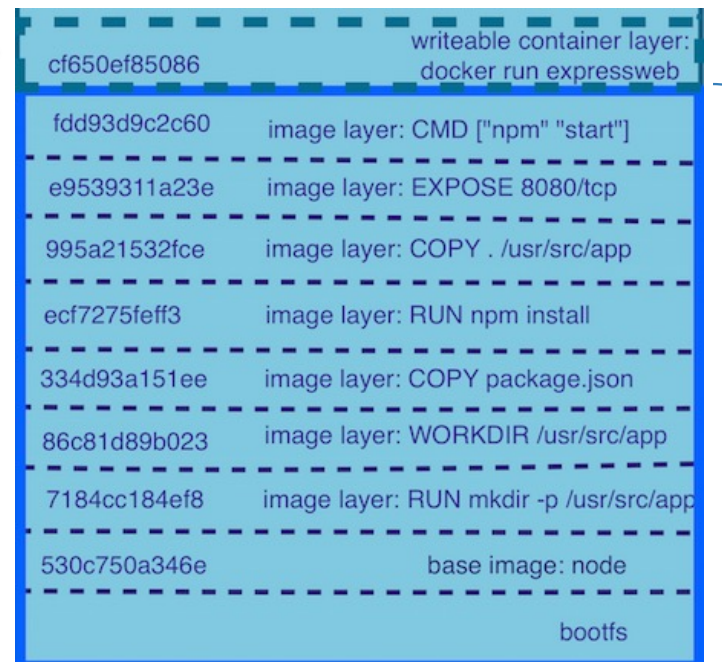
```
$ docker build -t expressweb .
Step 1 : FROM node:argon
argon: Pulling from library/node...
...
Status: Downloaded newer image for node:argon
---> 530c750a346e
Step 2 : RUN mkdir -p /usr/src/app
---> Running in 5090fde23e44
---> 7184cc184ef8
Removing intermediate container 5090fde23e44
Step 3 : WORKDIR /usr/src/app
---> Running in 2987746b5fba
---> 86c81d89b023
Removing intermediate container 2987746b5fba
Step 4 : COPY package.json /usr/src/app/
---> 334d93a151ee
Removing intermediate container a678c817e467
Step 5 : RUN npm install
---> Running in 31ee9721ccch
---> ecf7275feff3
Removing intermediate container 31ee9721ccch
Step 6 : COPY . /usr/src/app
---> 995a21532fce
Removing intermediate container a3b7591bf46d
Step 7 : EXPOSE 8080
---> Running in fddb8afb98d7
---> e9539311a23e
Removing intermediate container fddb8afb98d7
Step 8 : CMD npm start
---> Running in a262fd016da6
---> fdd93d9c2c60
Removing intermediate container a262fd016da6
Successfully built fdd93d9c2c60
```



Imagine

Containers layers

```
$ docker run expressweb  
...
```

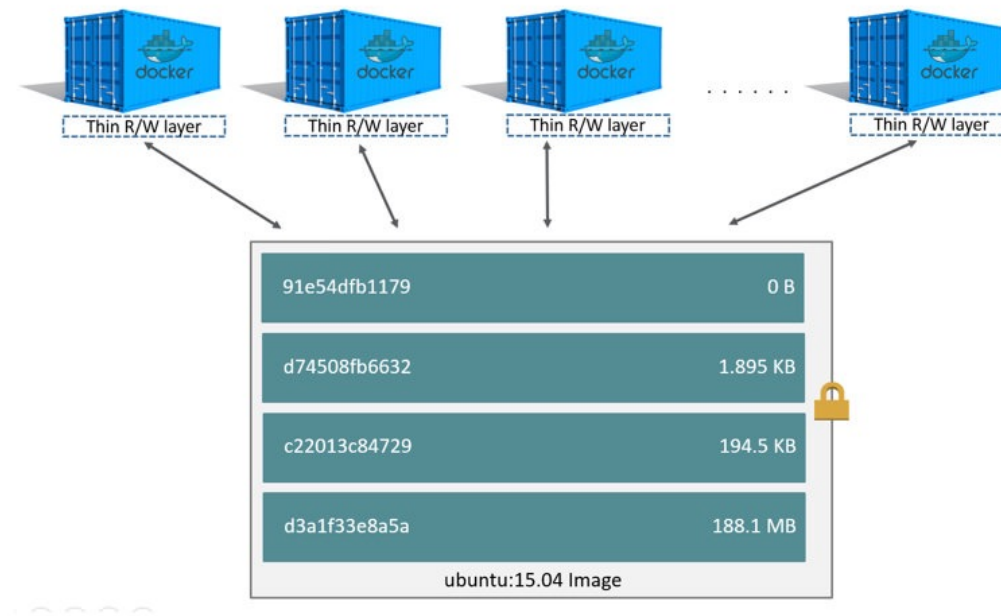


Container

Imagine

Containers layers

Più container creati dalla stessa immagine, condividono i layers di quest'ultima con un enorme risparmio di spazio disco.



Containers layers

```
$ docker ps -as
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES	SIZE
05f427a92591	nginx:1.17.9	"/bin/bash -c 'sleep...'"	12 days ago	Exited (1) 12 days ago		nginx_1	0B (virtual 127MB)

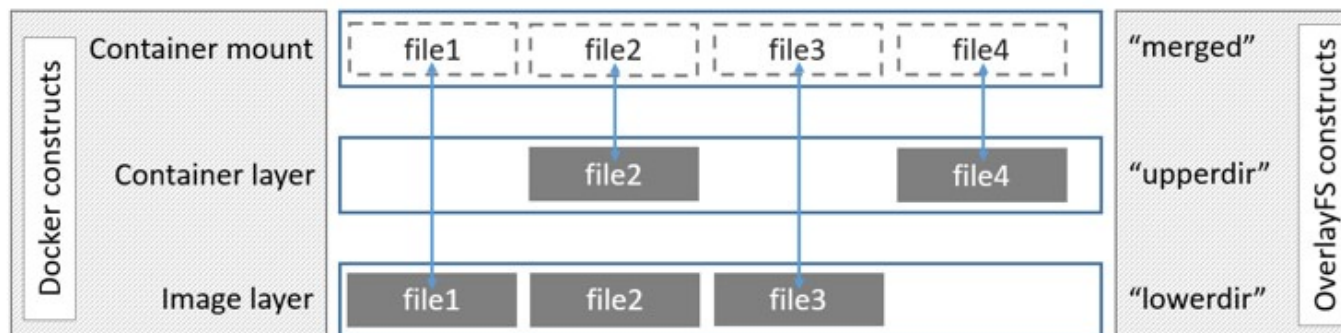
- **size** is the size on disk for the writable layer of the container; all data written in the container is stored there, and this is not shared between containers
- **virtual size** is the size of the "read-only" layer (the image that the container was started from) plus the size of the writable layer (size).

Copy-on-write strategy

Tutte le modifiche apportate dal container al filesystem sono memorizzate nel layer RW

Quando viene modificato un file esistente (presente nei layer RO dell'immagine), il driver di archiviazione esegue un'operazione di «copy-on-write». Per i driver aufs, overlay e overlay2, l'operazione di «copy-on-write» segue questa sequenza:

- Ricerca tra i livelli dell'immagine del file da modificare. Il processo inizia dal livello più recente (top) fino al livello base (down), un livello alla volta.
- Esegue un'operazione di «copy» sulla prima occorrenza del file trovato, per copiarlo nel livello RW del container.
- Le modifiche vengono apportate a questa copia del file e da quel momento il container non può vedere la copia del file esistente nel livello inferiore (RO).



Dati all'interno dei container

```
$ docker run -dit --name my_container_1 acme/my-final-image:1.0 bash \
&& docker run -dit --name my_container_2 acme/my-final-image:1.0 bash
```

```
38fa94212a419a082e6a6b87a8e2ec4a44dd327d7069b85892a707e3fc818544
1a174fc216cccf18ec7d4fe14e008e30130b11ede0f0f94a87982e310cf2e765
```

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
1a174fc216cc	acme/my-final-image:1.0	"bash"	About a minute ago	Up About a minute my_container_2
38fa94212a41	acme/my-final-image:1.0	"bash"	About a minute ago	Up About a minute my_container_1

```
$ sudo du -sh /var/lib/docker/containers/*
```

```
32K /var/lib/docker/containers/1a174fc216cccf18ec7d4fe14e008e30130b11ede0f0f94a87982e310cf2e765
32K /var/lib/docker/containers/38fa94212a419a082e6a6b87a8e2ec4a44dd327d7069b85892a707e3fc818544
```

Dati all'interno dei container

Il layer RW dei container, inizialmente è vuoto ed occupa all'incirca 32KB.

La strategia Copy-on-write non solo è più efficiente in termini di spazio, ma permette anche al container di essere veloce allo startup e shutdown

Se Docker dovesse creare un'intera copia dei livelli dell'immagine sottostante nel layer RW del container ogni volta che lo avvia, i tempi di startup e lo spazio su disco utilizzati aumenterebbero in modo significativo

QUINDI: I container non devono scrivere nel loro layer RW ma mantenerlo sempre di piccole dimensioni. I container devono persistere i dati ESCLUSIVAMENTE nei volumi (lezione successiva)

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Lab 10 – FS layering

Software Developer
Unità Formativa (UF): Containers - Docker
Docente: Denis Maggiorotto
Titolo argomento: Bind mount e volumi

Bind mount e volumi



Bind mount e volumi

Di default i container scrivono i file all'interno del loro RW layer, questo è sbagliato perché:

- I dati verrebbero persi se il container viene cancellato

- E' difficile portare i dati al di fuori del container

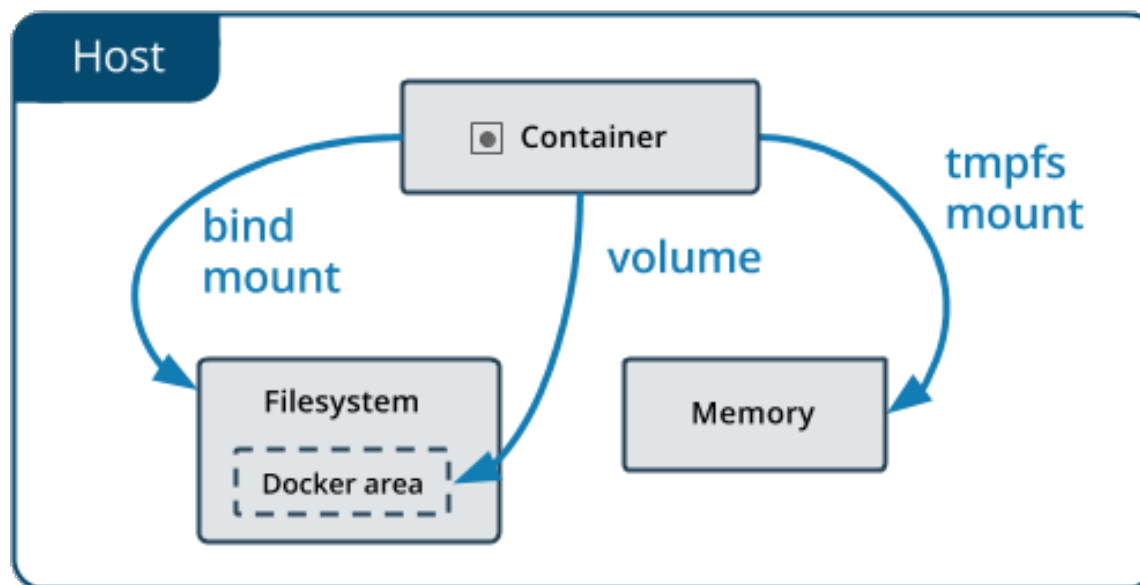
- Il layer RW risiede su di un singolo docker host, la soluzione non si presta se più container distribuiti su host diversi devono aver visibilità degli stessi dati

- Il layer RW di un container ha performance di scrittura e lettura pessime

- Scrivere nel layer RW di un container vuol dire farlo aumentare di dimensioni, il che rende più difficoltoso lo start e stop dello stesso

Docker e la persistenza dei dati

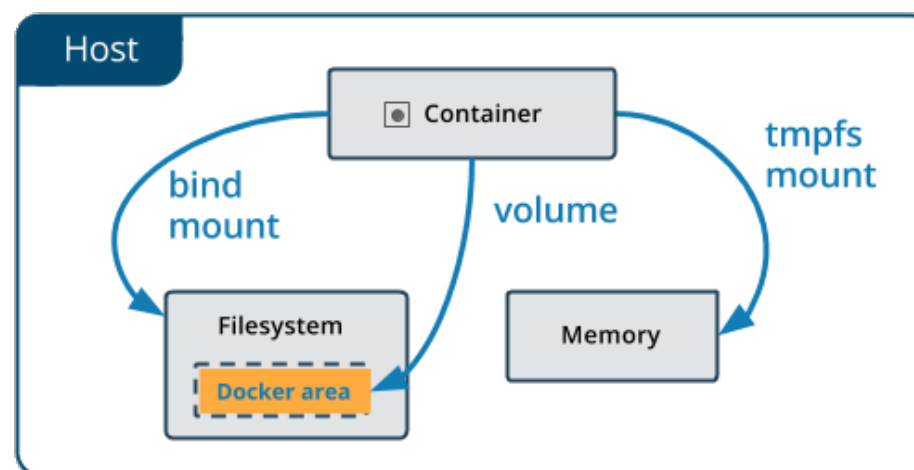
Docker offre tre funzionalità per la persistenza dei dati:



Volumi

I volumi sono la modalità consigliata per persistere dei dati da parte di un container, essi :

- Sono esterni al container, il quale li vede come fossero dei «dischi di rete»
- Sono facili da backuppare e ripristinare
- Vengono gestiti come un qualunque «oggetto Docker» tramite la Docker CLI
- Funzionano su Windows, Linux, Mac
- Possono essere condivisi tra più container



Volumi

Creazione di un volume

```
$ docker volume create my-vol
```

List dei volumi:

```
$ docker volume ls  
local          my-vol
```

Inspect di un volume:

```
$ docker volume inspect my-vol  
[  
  {  
    "Driver": "local",  
    "Labels": {},  
    "Mountpoint": "/var/lib/docker/volumes/my-vol/_data",  
    "Name": "my-vol",  
    "Options": {},  
    "Scope": "local"  
  }  
]
```

Rimozione di un volume

```
$ docker volume rm my-vol
```

Volumi

```
# --mount
$ docker run -d \
    --name devtest \
    --mount source=my-vol,target=/app \
    nginx:latest
```

Start di un container e mount di un volume con **--mount**

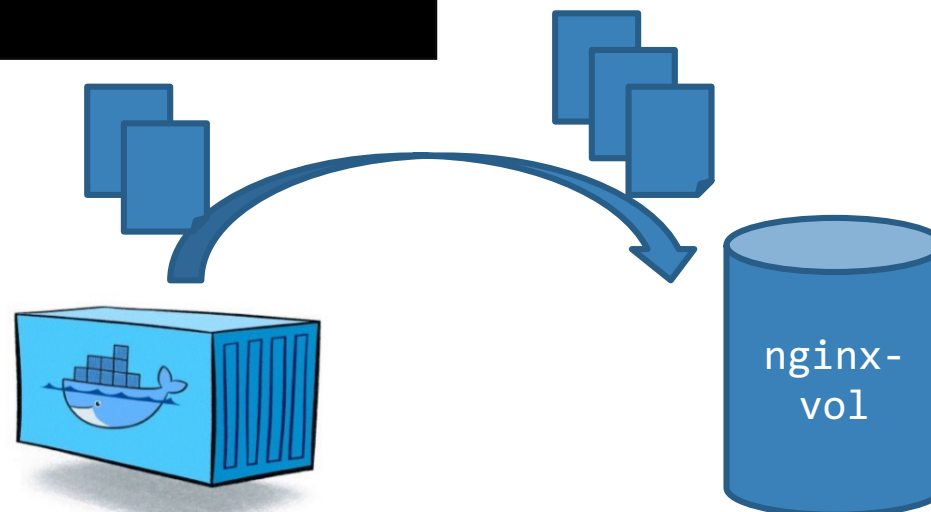
```
# -v
$ docker run -d \
    --name devtest \
    -v my-vol:/app \
    nginx:latest
```

Start di un container e mount di un volume con **-v**

Popolazione di un volume con un container

```
# --mount
$ docker run -d \
    --name=nginxtest \
    --mount source=nginx-
vol,destination=/usr/share/nginx/html \
    nginx:latest
```

```
# -v
$ docker run -d \
    --name=nginxtest \
    -v nginx-
vol:/usr/share/nginx/html \
    nginx:latest
```

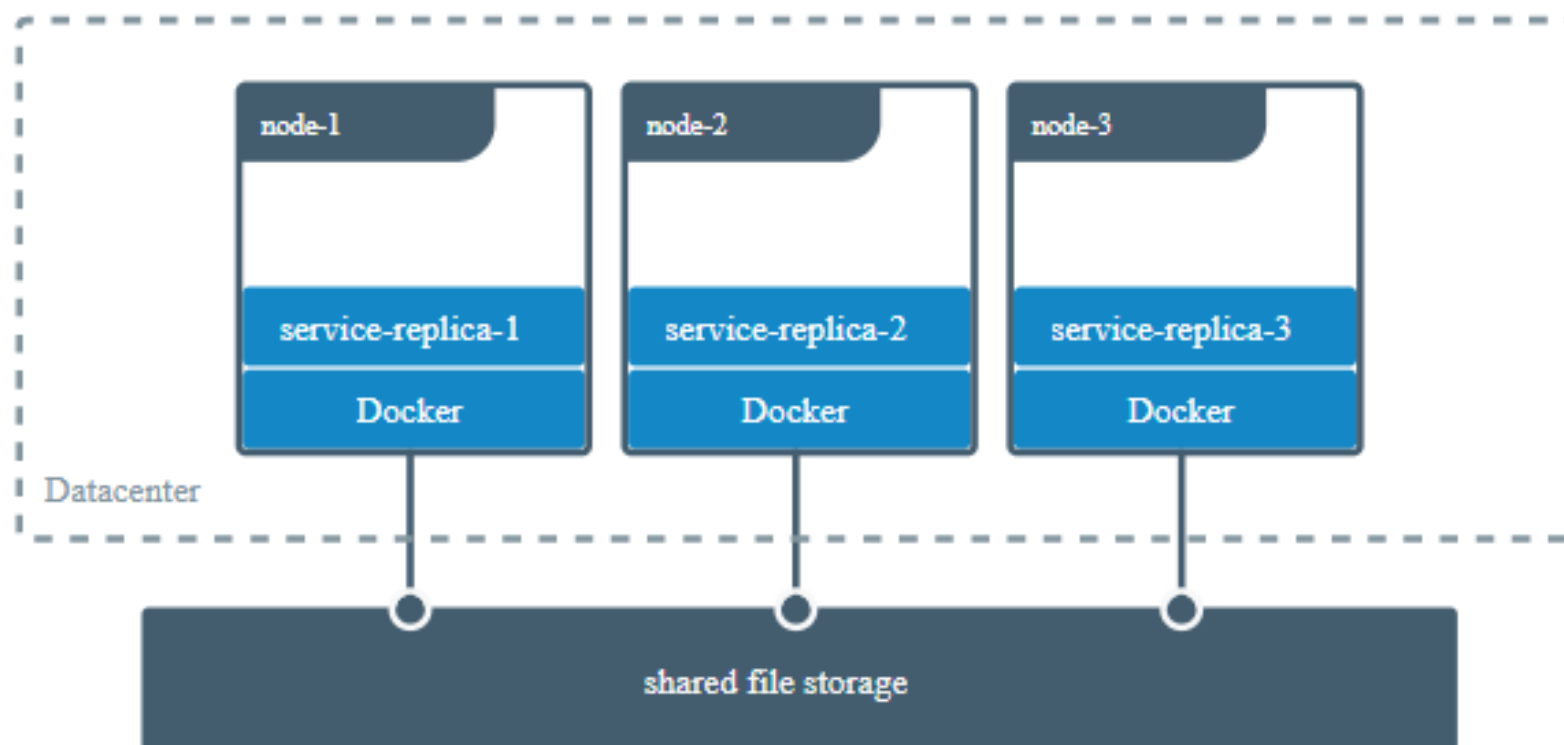


Volumi read-only

```
# --mount
$ docker run -d \
  --name=nginxtest \
  --mount source=nginx-vol,destination=/usr/share/nginx/html,readonly \
  nginx:latest
```

```
# -v
$ docker run -d \
  --name=nginxtest \
  -v nginx-vol:/usr/share/nginx/html:ro \
  nginx:latest
```

Volumi condivisi



Volumi condivisi

```
$ docker plugin install --grant-all-permissions vieux/sshfs
```

```
$ docker volume create --driver vieux/sshfs \  
  -o sshcmd=test@node2:/home/test \  
  -o password=testpassword \  
  sshvolume
```

```
$ docker run -d \  
  --name sshfs-container \  
  --volume-driver vieux/sshfs \  
  --mount src=sshvolume,target=/app,volume-opt=sshcmd=test@node2:/home/test,volume-opt=password=testpassword \  
  nginx:latest
```

Rimozione di volumi

Rimozione di un volume

```
$ docker volume rm my-vol
```

Rimozione di tutti i volume non utilizzati

```
$ docker volume prune
```

Bind mount

Le bind mount sono:

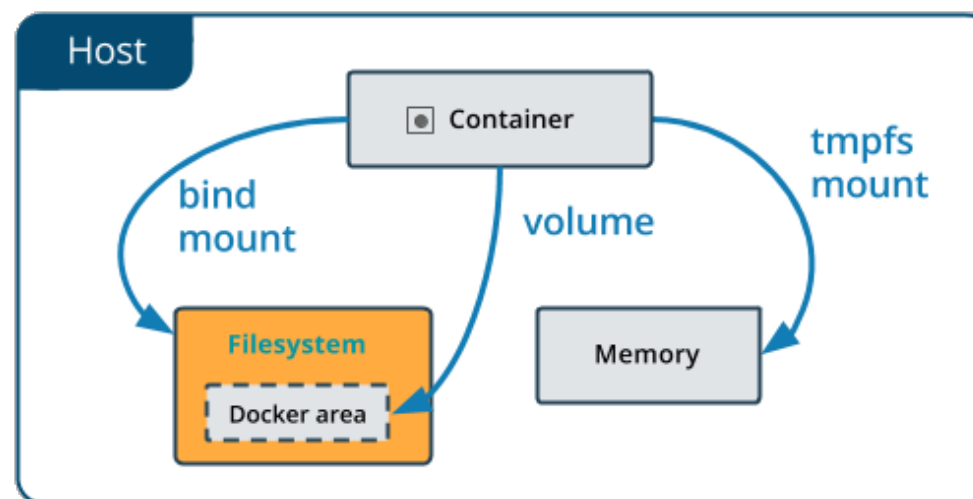
La condivisione di aree di file system del Docker host con il container

Normalmente sconsigliate al fine di persistere dei dati da parte del container

Meno sicure che i volumi

Difficili da utilizzare per condividere i dati tra container in esecuzione su Docker host differenti

Non portabili tra SO differenti



Start di un container con bind mount

```
# --mount
$ docker run -d \
    -it \
    --name devtest \
    --mount type=bind,source="$(pwd)"/target,target=/app \
    nginx:latest

# -v
$ docker run -d \
    -it \
    --name devtest \
    -v "$(pwd)"/target:/app \
    nginx:latest
```

Mount su directory piena

Nasconde il contenuto della directory sul container, a volte il container non funziona più

```
# --mount
$ docker run -d \
    -it \
    --name broken-container \
    --mount type=bind,source=/tmp,target=/usr \
    nginx:latest

docker: Error response from daemon: oci runtime error:
container_linux.go:262: starting container process caused "exec:
\"nginx\": executable file not found in $PATH".
```

Read-only bind mount

```
# --mount
$ docker run -d \
    -it \
    --name devtest \
    --mount type=bind,source="$(pwd)"/target,target=/app,readonly \
    nginx:latest

# -v
$ docker run -d \
    -it \
    --name devtest \
    -v "$(pwd)"/target:/app:ro \
    nginx:latest
```

tmpfs

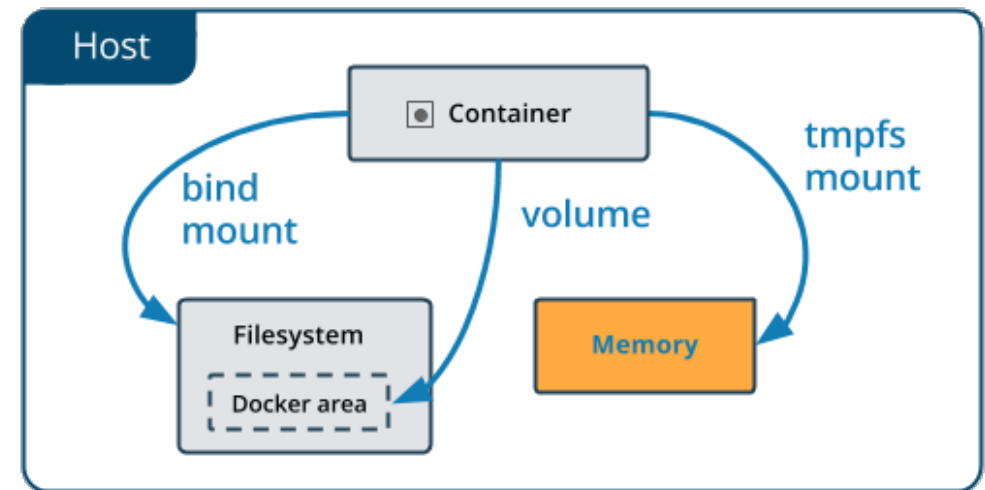
Le volumi tmpfs sono:

Effimeri, perdono il contenuto allo stop del container

Veloci in scrittura

Utilizzati quando il container deve produrre una grande quantità di dati, non particolarmente utili

Funzionano solo su Docker host linux



tmpfs

```
# --mount  
$ docker run -d \  
    -it \  
    --name tmptest \  
    --mount type=tmpfs,destination=/app \  
    nginx:latest
```

```
# --tmpfs  
$ docker run -d \  
    -it \  
    --name tmptest \  
    --tmpfs /app \  
    nginx:latest
```

Opzioni tmpfs

Option	Description
tmpfs-size	Size of the tmpfs mount in bytes. Unlimited by default.
tmpfs-mode	File mode of the tmpfs in octal. For instance, 700 or 0770. Defaults to 1777 or world-writable.

```
$ docker run -d \  
  -it \  
  --name tmptest \  
  --mount type=tmpfs,destination=/app,tmpfs=1770 \  
  nginx:latest
```

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Lab	11 – Volumes
Lab	12 – Bind mounts
Lab	13 – TmpFS mounts (solo per Linux e MacOS)
Assignment	03 – Manage backups using volumes